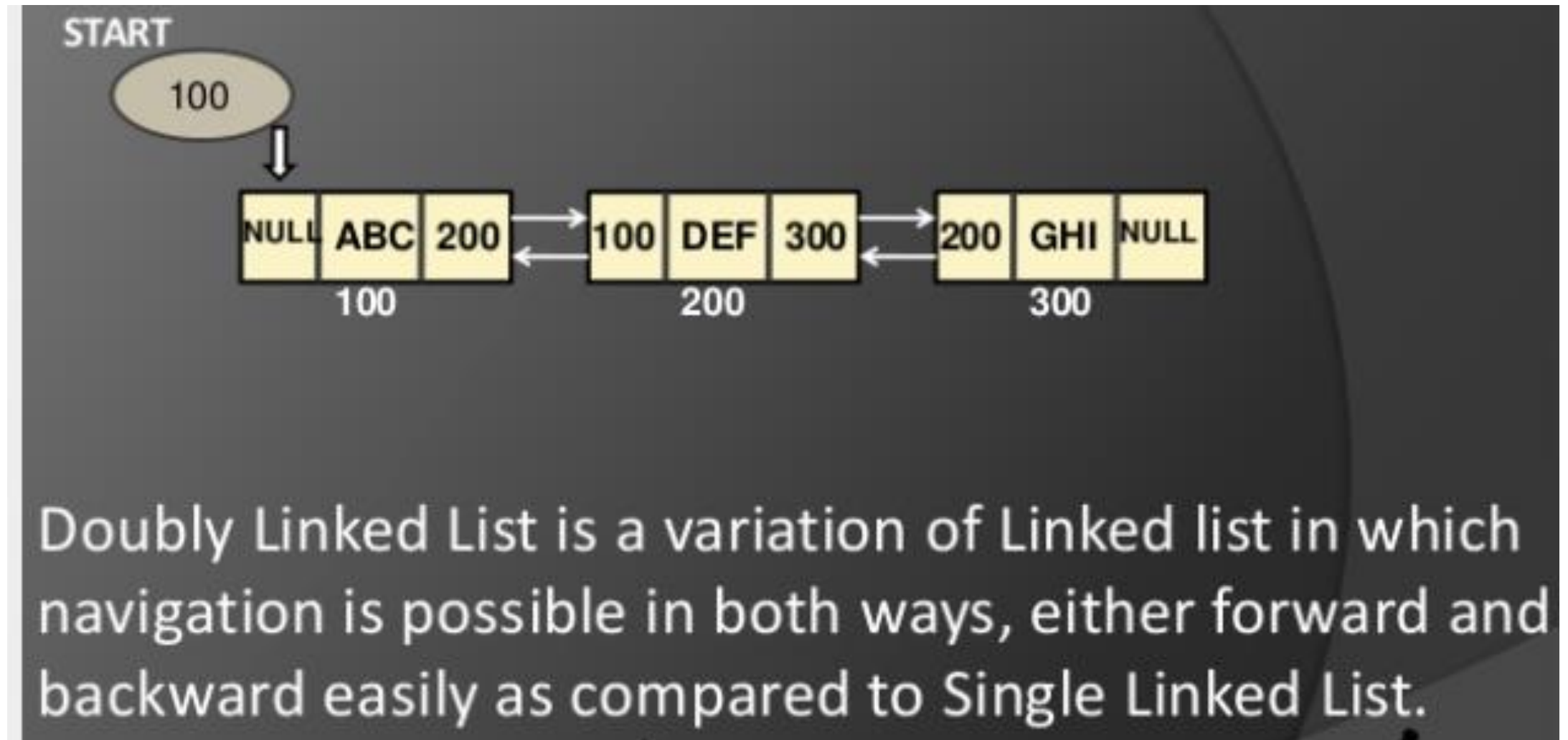


DOUBLY LINKED LIST OPERATIONS

Module 2

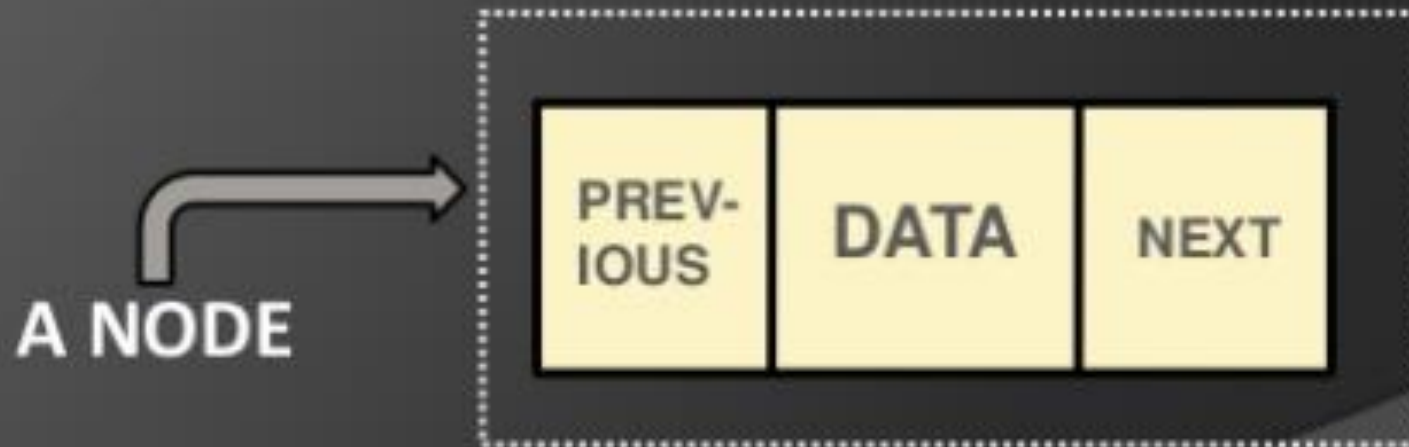


Doubly Linked List



Why Doubly Linked List

- In singly linked list we cannot traverse back to the previous node without an extra pointer. For ex to delete previous node.
- In doubly there is a link through which we can go back to previous node.



OPERATIONS ON DOUBLY LINK LIST

```
graph TD; A[OPERATIONS ON DOUBLY LINK LIST] --> B[INSERTION]; A --> C[DELETION]; A --> D[TRAVERSING]; B --> B1[• AT FIRST]; B --> B2[• AT LAST]; B --> B3[• AT DESIRED]; C --> C1[• AT FIRST]; C --> C2[• AT LAST]; C --> C3[• AT DESIRED]; D --> D1[• LOOKUP];
```

INSERTION

- AT FIRST
- AT LAST
- AT DESIRED

DELETION

- AT FIRST
- AT LAST
- AT DESIRED

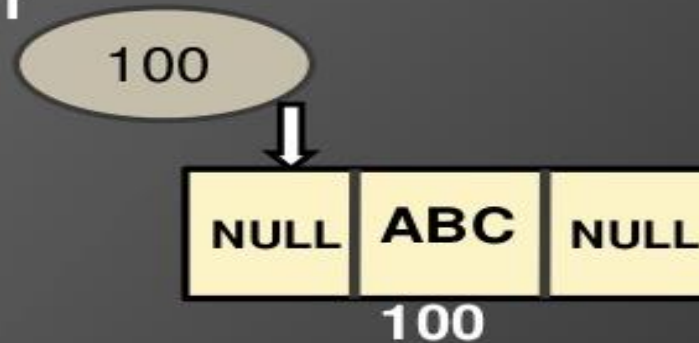
TRAVERSING

- LOOKUP

Create

CREATING A LIST WHEN THERE ARE NO ELEMENTS.

START



ALGO :-

STEP 1 : BEGIN.

STEP 2 : CREATE A NODE(NEWNODE)

NEWNODE -> DATA = VALUE.

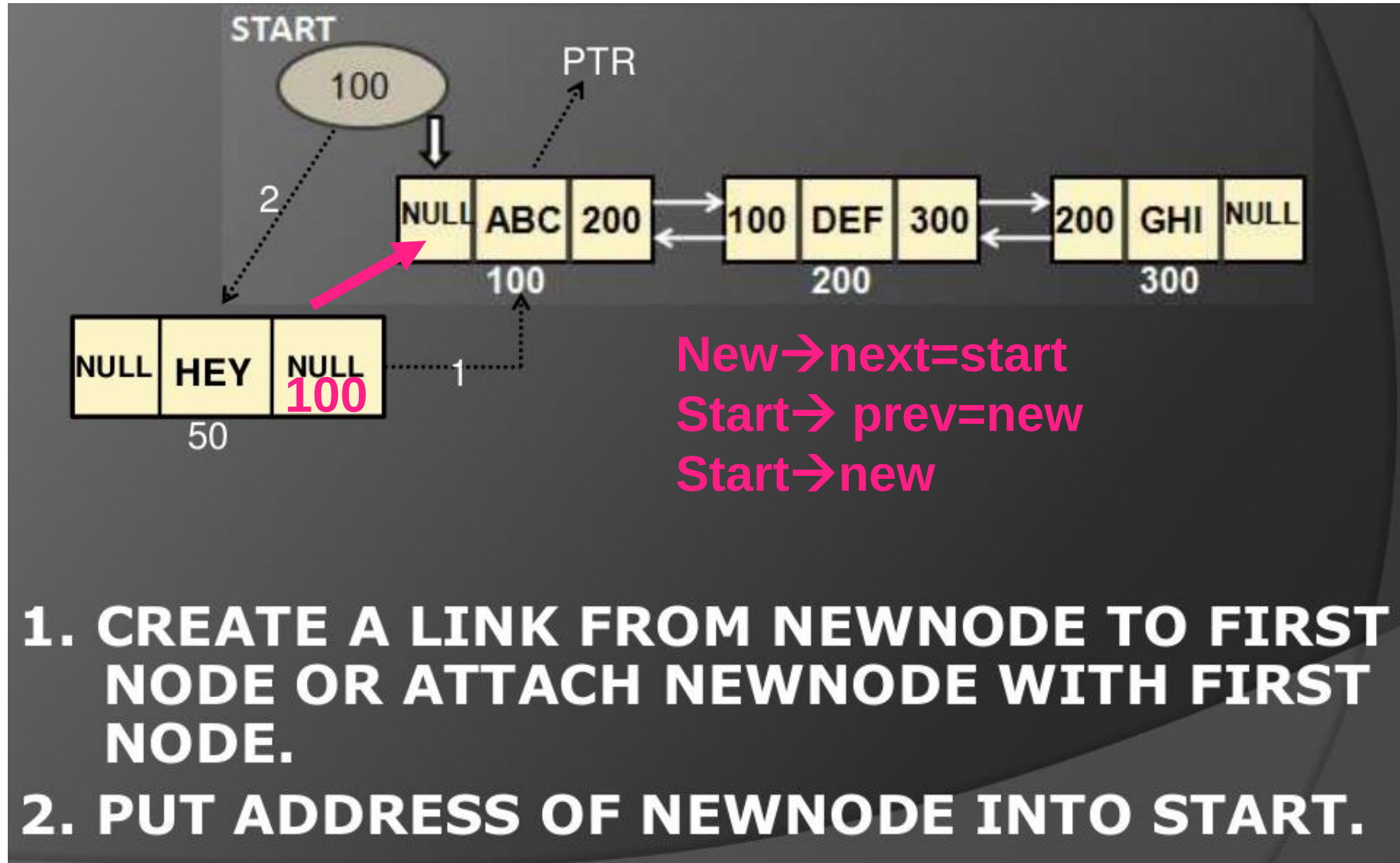
NEWNODE -> PREVIOUS = NULL.

NEWNODE -> NEXT = NULL.

STEP 3 : START=NEWNODE.

STEP 4 : EXIT.

Insertion at beginning



Algorithm

STEP 1 : BEGIN.

**STEP 2 : CREATE A NODE(NEWMODE)
NEWMODE -> DATA = VALUE.
NEWMODE -> PREVIOUS = NULL.
NEWMODE ->NEXT = NULL.**

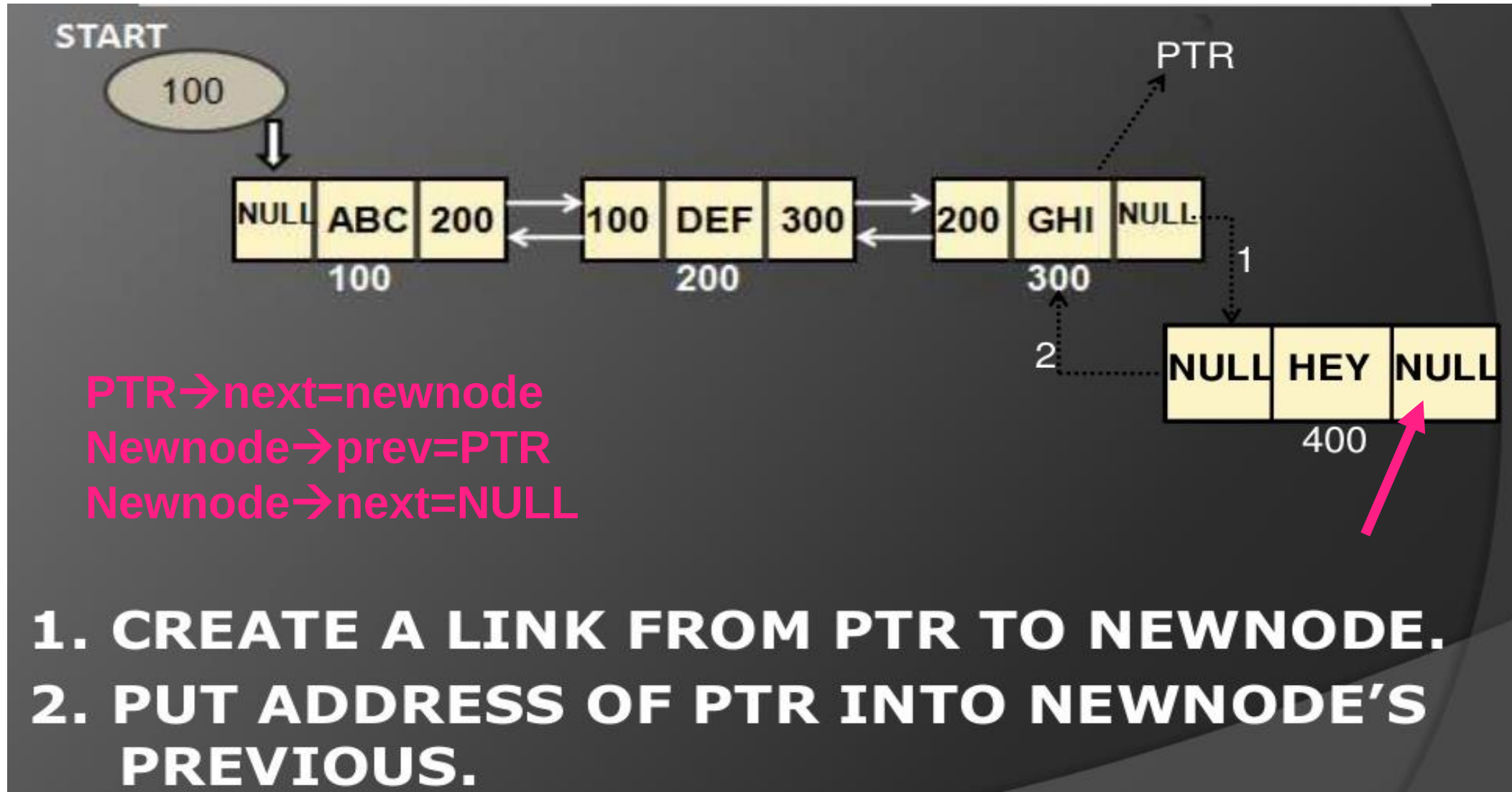
STEP 3 : NEWMODE -> NEXT = START.

STEP 4 : PTR ->PREV = NEWMODE.

STEP 5 : START = NEWMODE.

STEP 6 : EXIT.

Insertion at the Last



Algorithm

STEP 1 : BEGIN.

STEP 2 : CREATE A NODE(NEWNODE)
NEWNODE -> DATA = VALUE.
NEWNODE -> PREVIOUS = NULL.
NEWNODE -> NEXT = NULL.

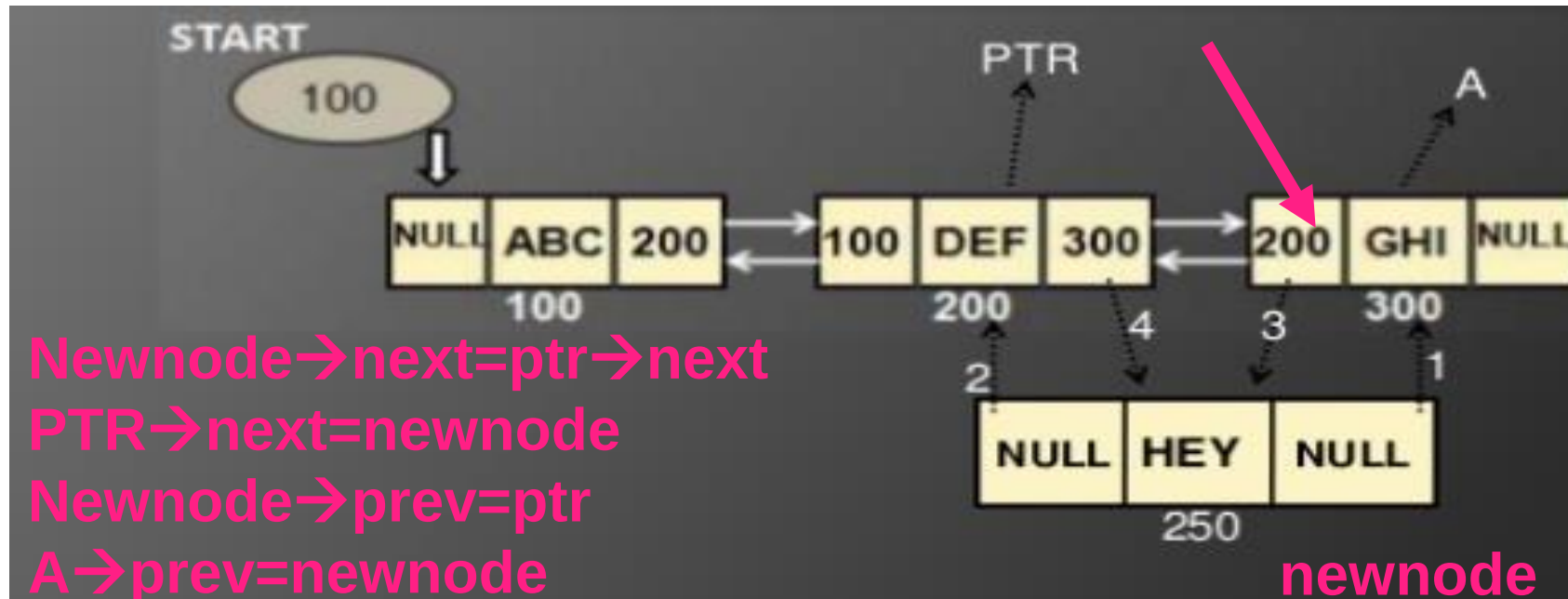
STEP 3 : PTR = START.
WHILE(PTR -> NEXT != NULL)
{
PTR = PTR -> NEXT.
}

STEP 4 : PTR -> NEXT = NEWNODE.

STEP 5 : NEWNODE -> PREV = PTR.

STEP 6 : EXIT.

Insertion at desired location in the list



- 1 CONNECT NEWNODE WITH A .
- 2 CONNECT NEWNODE WITH PTR.
- 3 CONNECT A WITH NEWNODE.
- 4 CONNECT PTR WITH NEWNODE.

Algorithm

STEP 1 : BEGIN.

STEP 2 : CREATE A NODE(NEWMODE)

NEWMODE -> DATA = VALUE.

NEWMODE -> PREVIOUS = NULL.

NEWMODE -> NEXT = NULL.

STEP 3 : PTR = START.

WHILE(PTR -> NEXT != CHOICE)

{

PTR = PTR -> NEXT.

}

STEP 4 : NEWMODE -> NEXT = A{PTR->NEXT}.

STEP 5 : NEWMODE -> PREV = PTR.

STEP 6 : A -> PREV = NEWMODE.

STEP 7 : PTR -> NEXT = NEWMODE.

STEP 8 : EXIT.

Deletion

WHEN THERE ARE NO
ELEMENTS



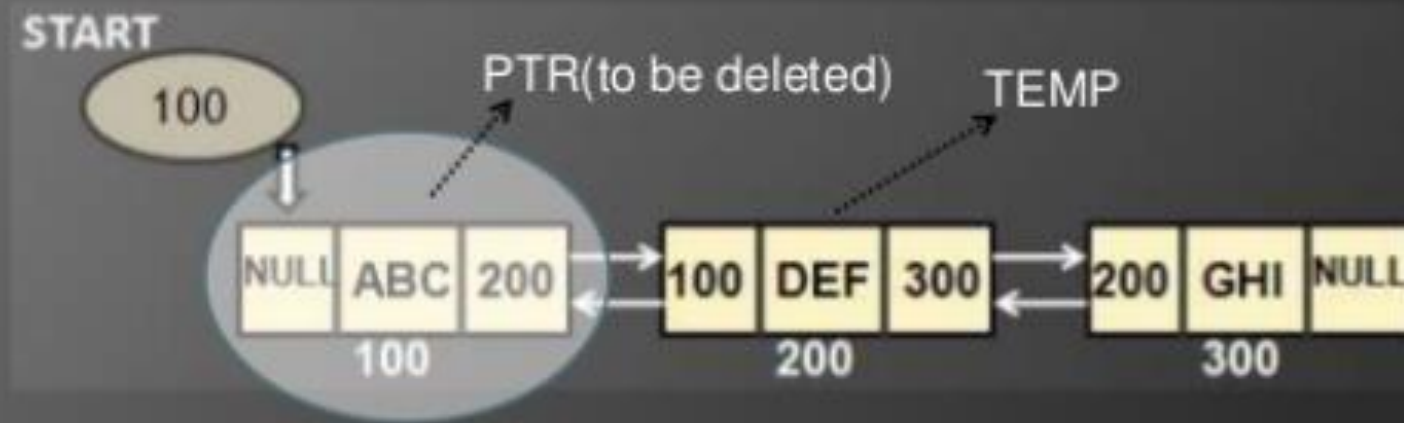
ALGO :-

STEP 1 : BEGIN.

STEP 2 : IF $START == NULL$ THEN PRINT LIST IS EMPTY

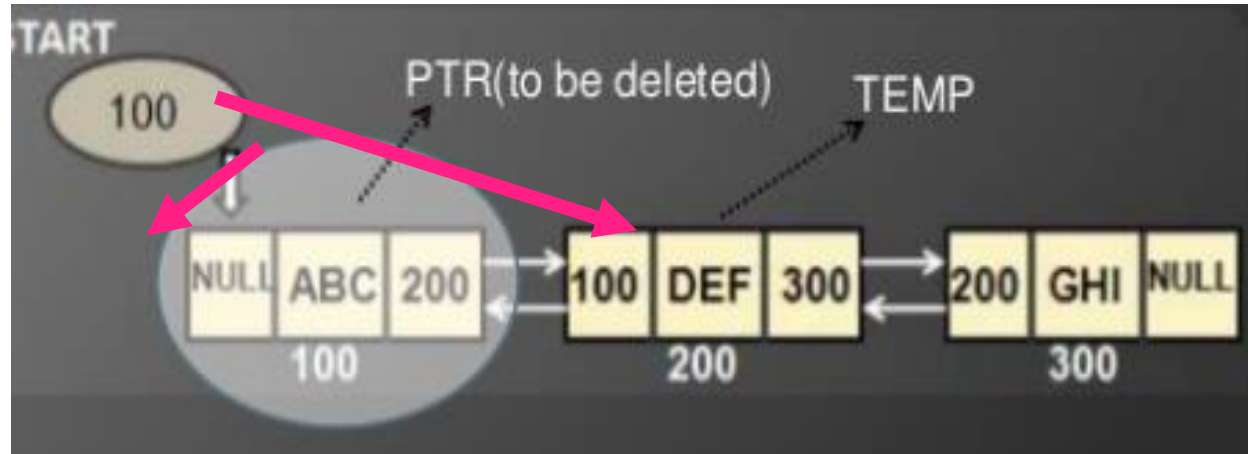
STEP 3 : EXIT.

DELETION AT FIRST



- 1 CONNECT START WITH SECOND NODE.
- 2 PUT NULL IN PREVIOUS OF SECOND NODE.
- 3 free up the space occupied by first node.

Algorithm



Start=200
Ptr=100

STEP 1 : BEGIN.

STEP 2 : PTR = START.

STEP 3 : START = PTR -> NEXT.

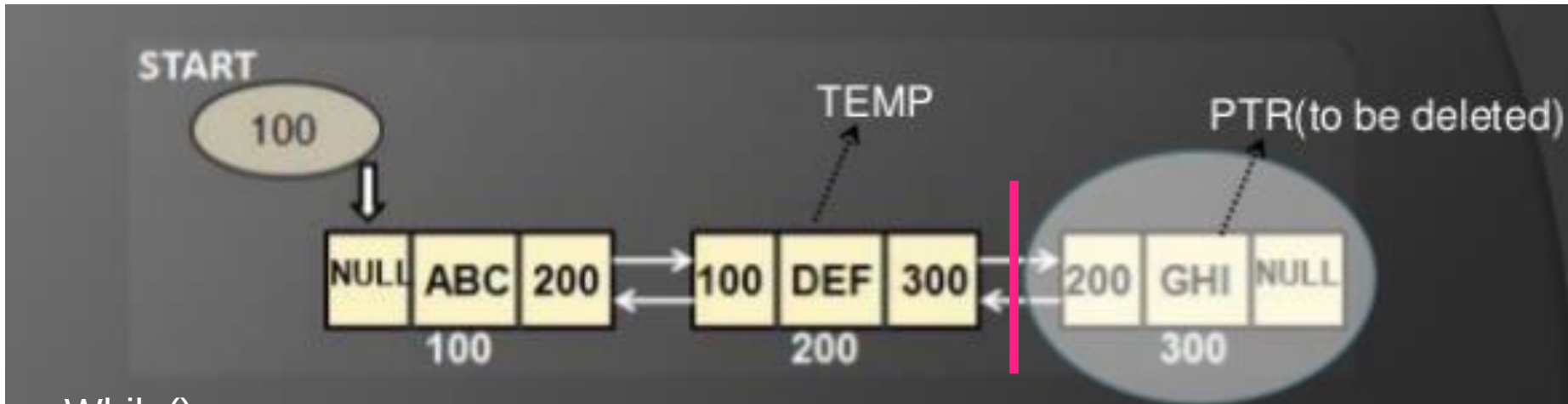
STEP 4 : TEMP -> PREV = NULL.

{TEMP = PTR->NEXT}

STEP 5 : FREE (PTR).

STEP 6 : EXIT.

Deletion at the last



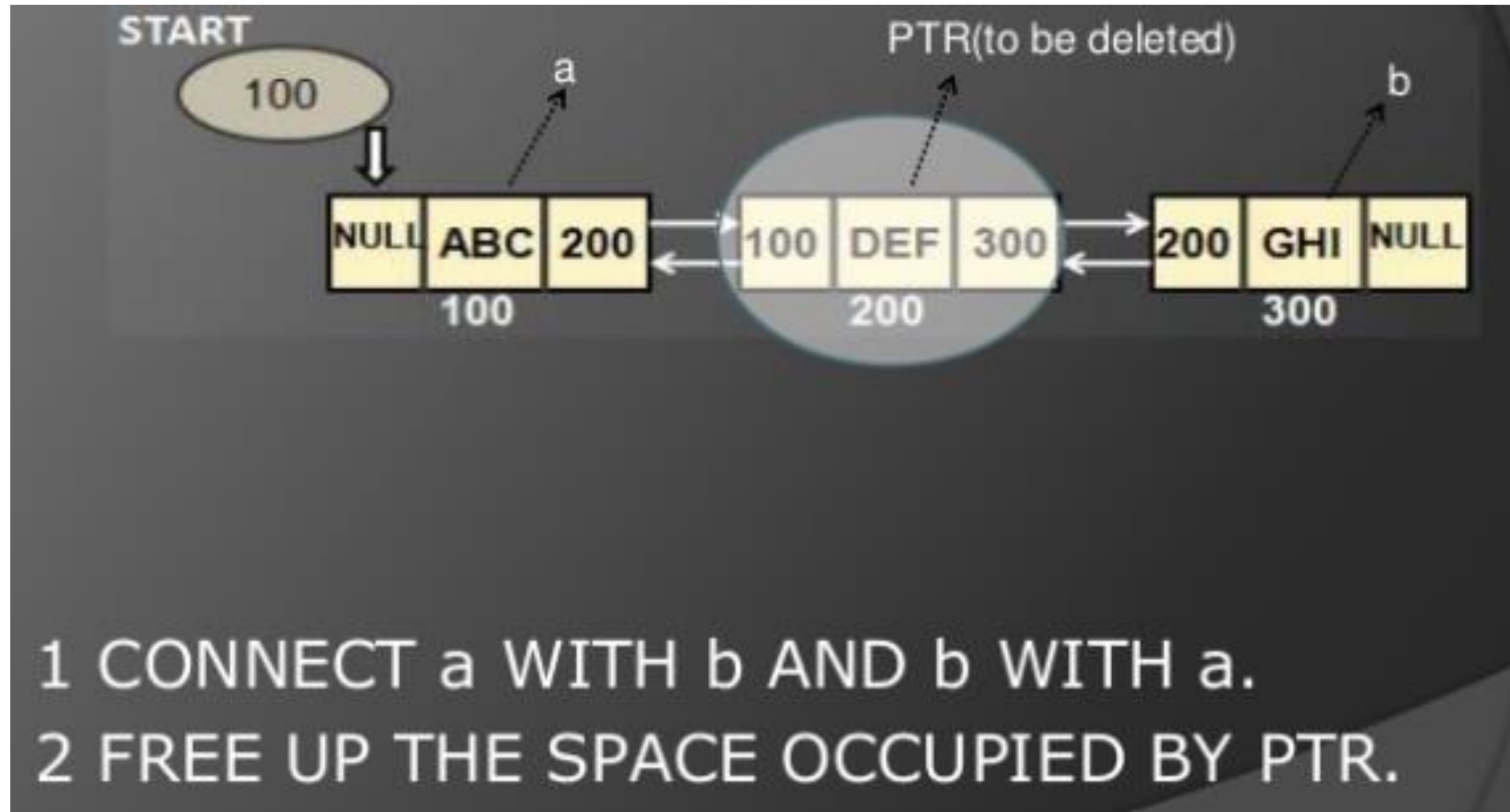
```
While()  
PTR=300  
TEMP=PTR→prev  
TEMP→NEXT=NULL  
Free(PTR)
```

- 1 Put null in next part of second last node.
- 2 free up the space occupied by last node.

Algorithm

```
STEP 1 : BEGIN.  
STEP 2 : PTR = START.  
STEP 3 : WHILE(PTR -> NEXT != NULL)  
        {  
            PTR= PTR -> NEXT;  
        }  
STEP 4 : TEMP = PTR -> PREV.  
STEP 5 : TEMP -> NEXT = NULL.  
STEP 6 : FREE (PTR).  
STEP 7 : EXIT.
```

Deletion at the desired location



Algorithm

```
STEP 1 : BEGIN.  
STEP 2 : PTR -> NEXT = b.  
STEP 3 : PTR -> PREV = a.  
STEP 2 : PTR = START.  
STEP 3 : WHILE(PTR -> NEXT != CHOICE)  
        {  
            PTR = PTR -> NEXT;  
        }  
STEP 4 : b -> PREV = a.  
STEP 5 : a -> NEXT = b.  
STEP 6 : FREE (PTR).  
STEP 7 : EXIT.
```

Traversing

